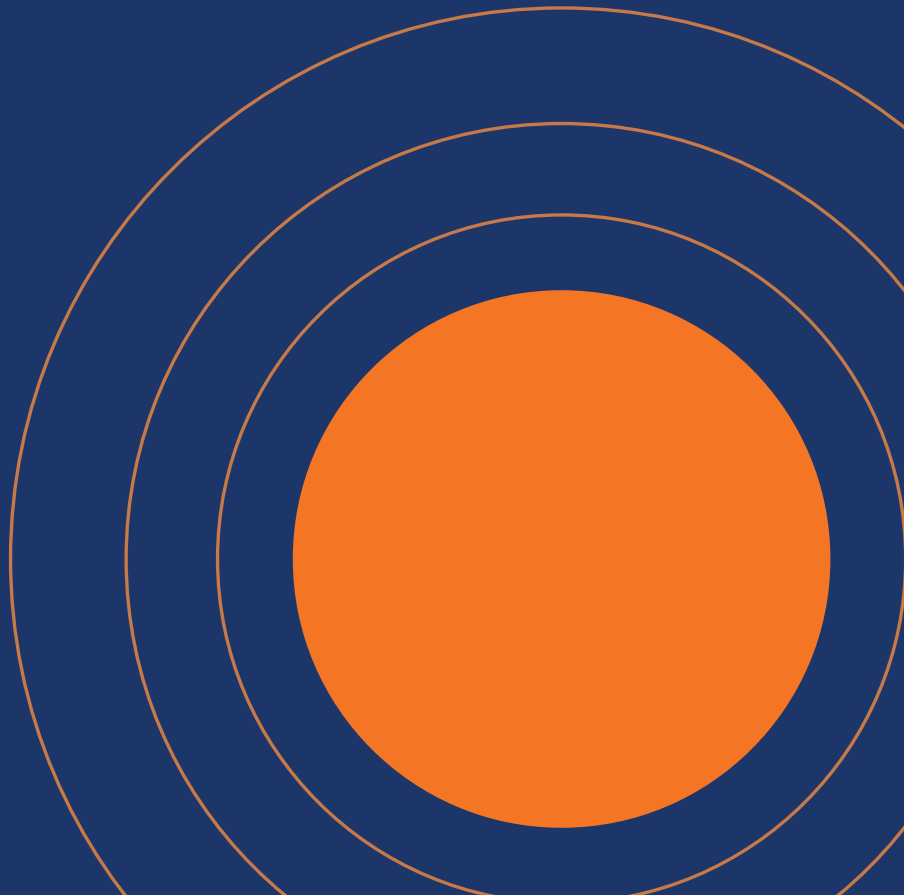
AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

UNCONSTRAINED OPTIMIZATION II

Newton's Method: Smarter Steps Using the Bend



Dr. Taha Yiğit Alkan · Department of Computer Engineering

Last Week Worked, But Left Two Problems

GRADIENT DESCENT (WEEK 5)

new x = x − α × slope

Uses only the SLOPE: which way is down.

ITS TWO PROBLEMS

1. You must guess the step size α.
2. Narrow valleys: 139 zig-zag steps.

Part I

Newton's idea: use the slope AND the bend.

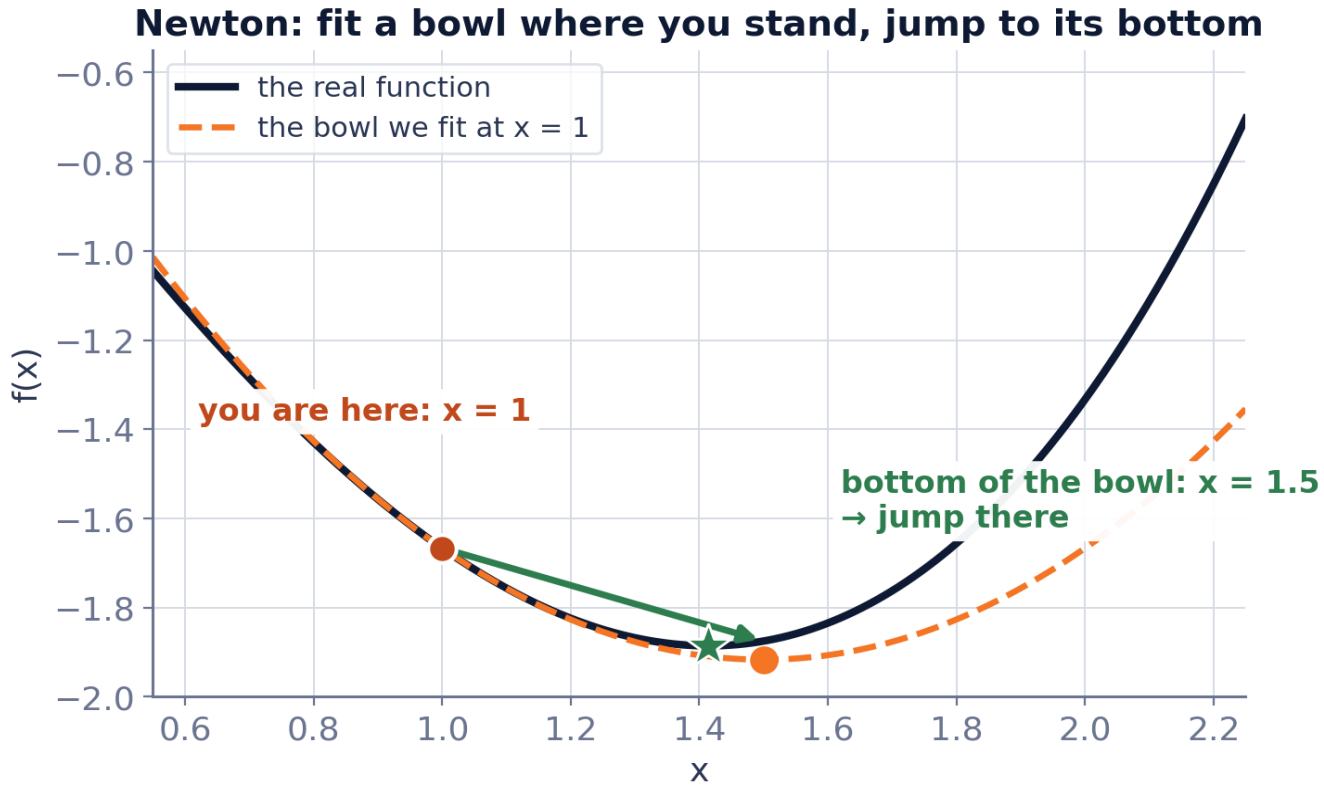
Part II

The race: how much faster is it?

Part III

The catches — and the clever compromise (BFGS).

Fit a Bowl Where You Stand, Then Jump to Its Bottom



GRADIENT DESCENT ASKS

Which way is down?
→ then guesses how far.

NEWTON ASKS

Which way is down, AND how far is the bottom?
→ it computes the step.

Newton's trick: replace the real function by a simple bowl that matches it here, and jump to that bowl's bottom.

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

The Bend: How Fast the Slope Changes

IN PLAIN WORDS

The slope says how steep the ground is. The bend says how quickly that steepness changes as you walk.

x	slope of $(x - 3)^2$	change from previous
7	8	—
6	6	drops by 2
5	4	drops by 2
4	2	drops by 2

WHAT THE TABLE SAYS

Every step of 1 to the left, the slope drops by 2.
So the bend of $(x - 3)^2$ is 2 — everywhere.

big bend: a sharp, narrow bowl

small bend: a wide, flat bowl

negative bend: the top of a hill (careful!)

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Newton's Method Is Also One Line

$$\text{new } x = \text{old } x - \text{slope} \div \text{bend}$$ (no step size to guess)

WORKED EXAMPLE

Week 5's bowl $(x - 3)^2$, starting at $x = 7$

slope = 8, bend = 2

step = slope ÷ bend = 8 ÷ 2 = 4

new x = 7 − 4 = 3 → the bottom, in ONE jump

gradient descent (α = 0.25): 7 → 5 → 4 → 3.5 → …

A CONNECTION TO WEEK 5

Last week we found by trial and error that the best step size for this bowl was α = 0.5.

Newton's step uses 1 ÷ bend = 1 ÷ 2 = 0.5 — the same number, computed instead of guessed.

On any bowl-shaped (quadratic) function, Newton lands on the bottom in exactly one jump.

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

How Your Calculator Finds √2

Find the lowest point of $f(x) = x^3/3 - 2x$ (for $x > 0$). **slope = $x^2 - 2$** **bend = $2x$** The answer is $x = \sqrt{2} = 1.41421356\ldots$

step	x	slope	bend	slope ÷ bend	new x
0	1	−1	2	−0.5	**1.5**
1	1.5	0.25	3	0.0833	**1.416667**
2	1.416667	0.00694	2.8333	0.00245	**1.414216**
3	1.414216	0.000006	2.8284	0.0000021	**1.41421356**

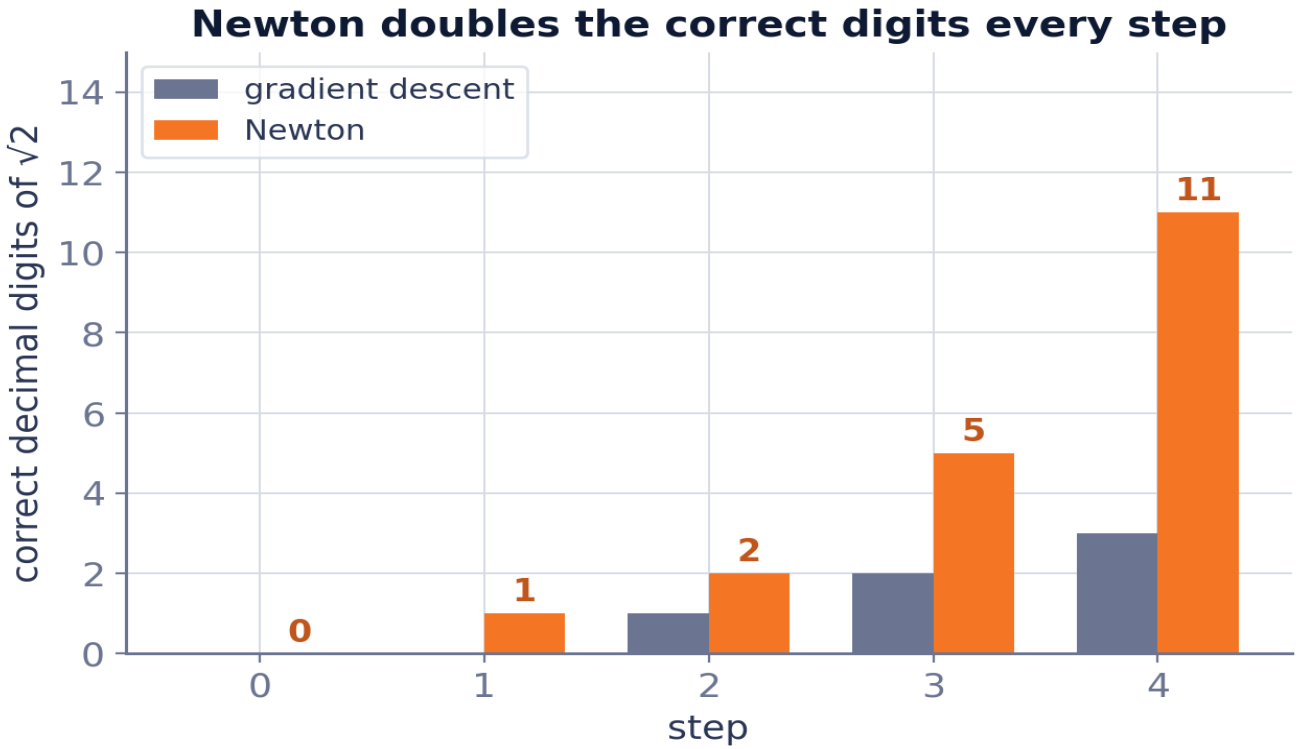
LOOK AT THE LAST COLUMN

After 4 steps:
1.41421356

That is √2 to 8 decimals — from a start of 1.

Simplified, the step is new $x = (x + 2/x) \div 2$ — the 'Babylonian' square-root rule, 4,000 years old.

Near the Answer, the Correct Digits Double



step	error of Newton
0	0.41
1	0.086
2	0.0025
3	0.0000021
4	0.0000000000016

PATTERN

The error is roughly SQUARED each step.

Gradient descent adds about one correct digit every few steps. Newton doubles them.

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Try It Yourself: Find √3

THE TASK

Lowest point of $f(x) = x^3/3 - 3x$.
slope $= x^2 - 3$, bend $= 2x$.
Start at $x = 2$. Do two Newton steps.

step	x	slope	bend	slope ÷ bend	new x
0	2				
1					

BONUS QUESTION

Start at $x = -1$ instead.

Where does Newton take you after two steps?

Is that point a valley bottom, or something else?

Hint: look at the sign of the bend there.

Answers

step	x	slope	bend	slope ÷ bend	new x
0	2	1	4	0.25	**1.75**
1	1.75	0.0625	3.5	0.0179	**1.7321**

CHECK

√3 = 1.7320508…
Two steps already give 1.7321 — four correct digits.

BONUS: START AT x = −1

x	slope	bend	new x
−1	−2	−2	**−2**
−2	1	−4	**−1.75**
−1.75	0.0625	−3.5	**−1.7321**

Newton found x = −√3, where the slope is zero — but the bend is NEGATIVE: it is a hilltop, not a valley.

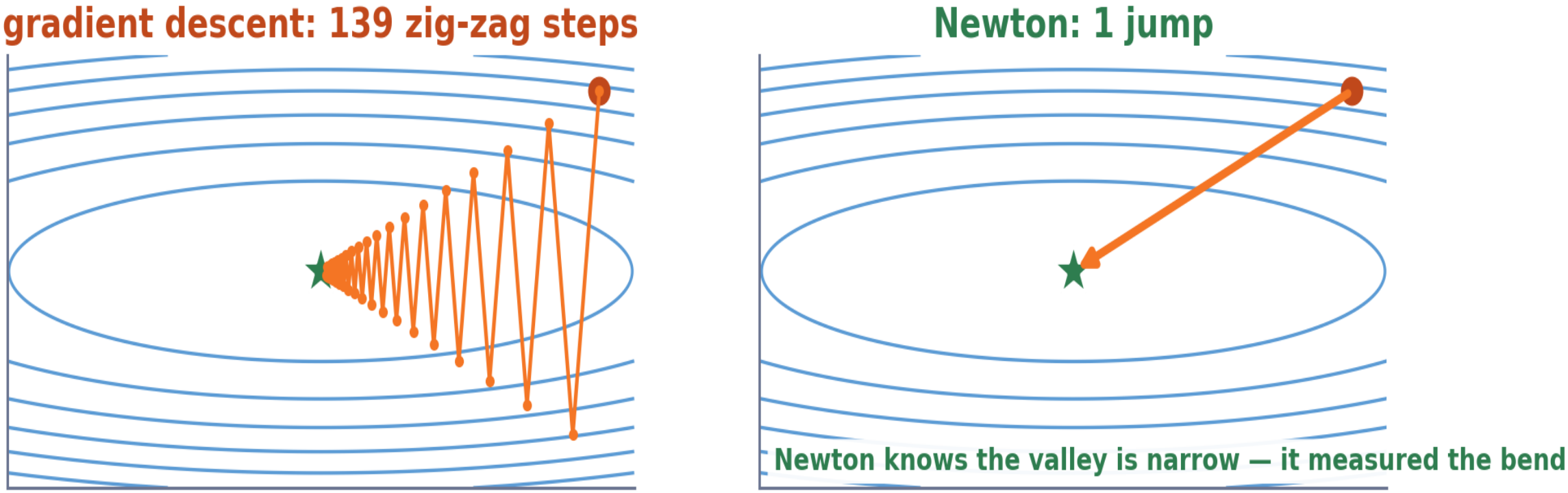
Newton looks for 'zero slope'. It does not check whether that is a valley or a hilltop.

PART II

The Race

Same problems as Week 5 — how much does knowing the bend help?

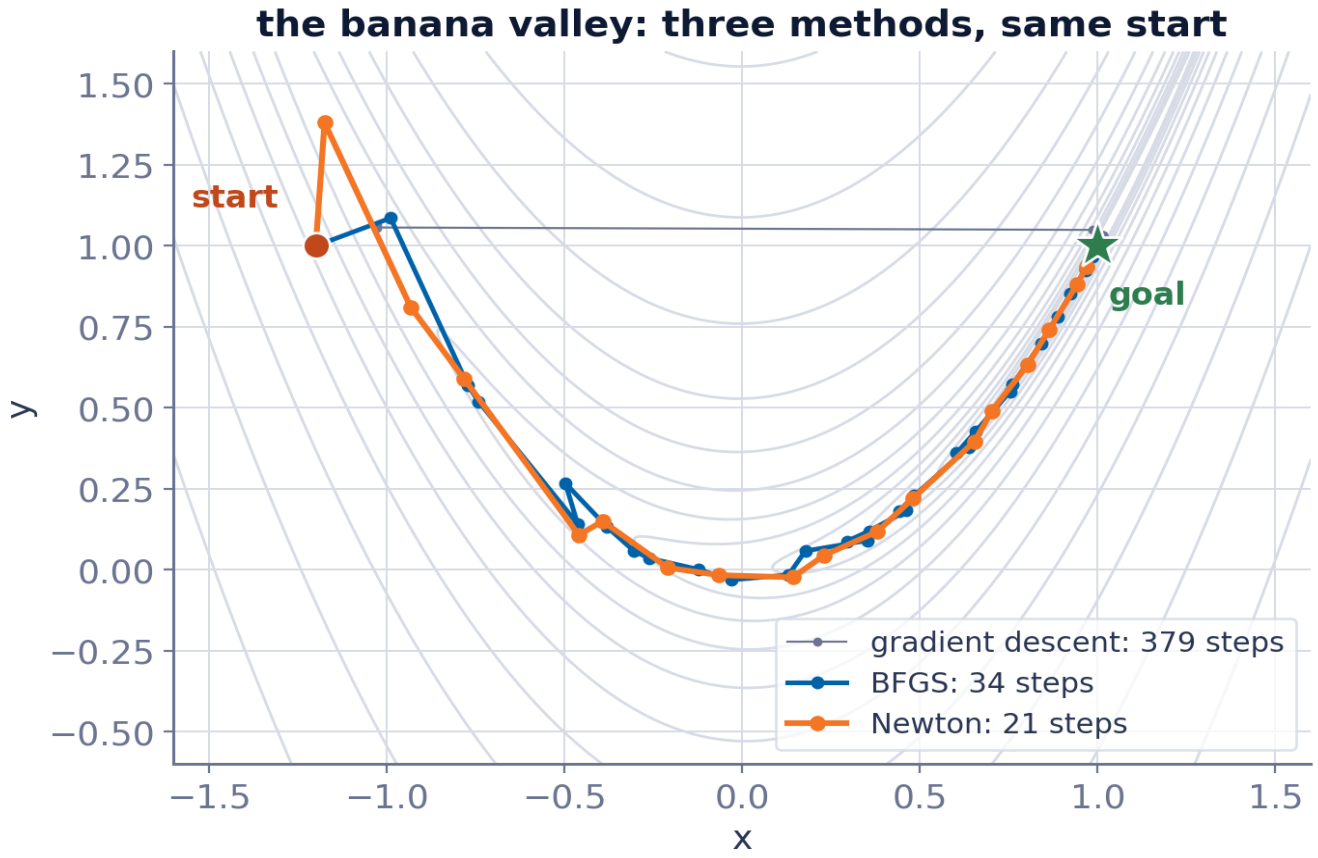
Narrow Valleys No Longer Matter



WHY

Newton measures the bend in EVERY direction, so it knows the valley is steep across and gentle along.

The Banana Valley: 379 vs 34 vs 21 Steps



method	uses	steps
gradient descent	slope	379
BFGS (Part III)	slope + learned bend	34
Newton	slope + exact bend	21

THE BANANA

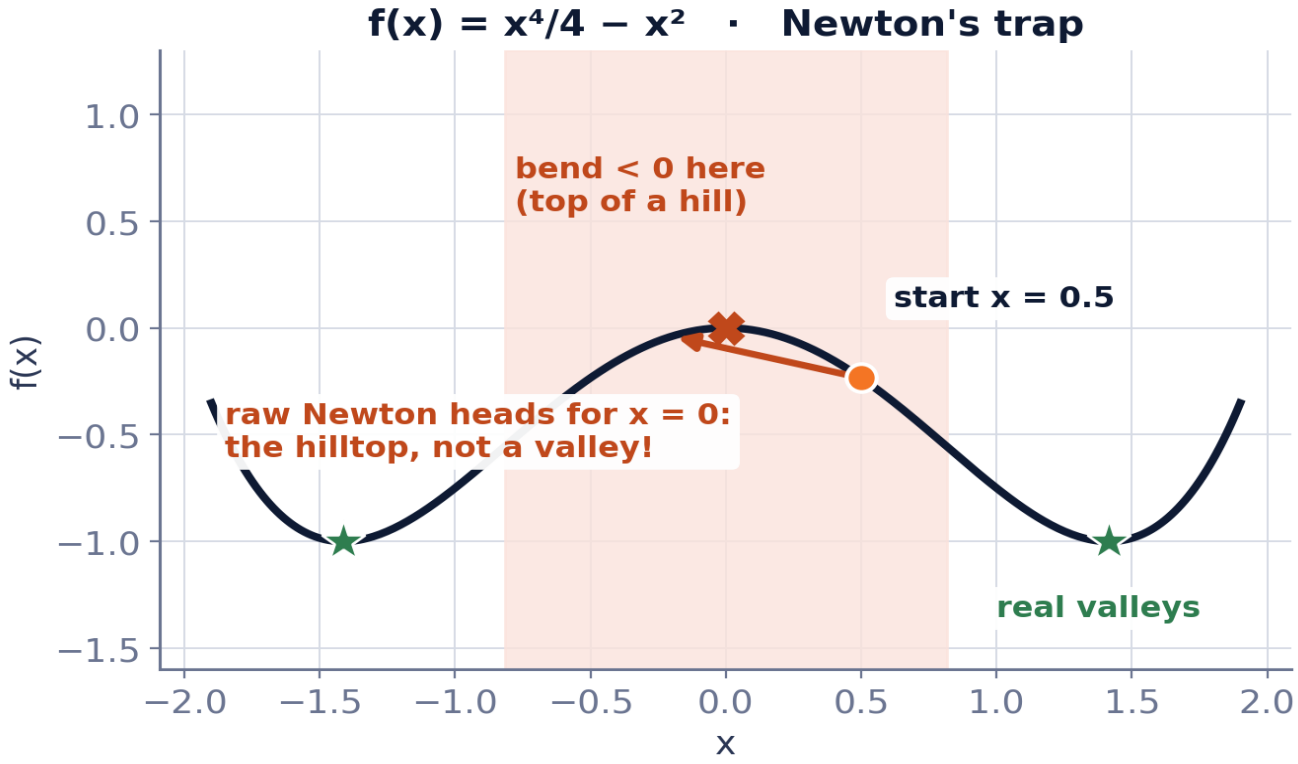
A famous test: a long, curved, narrow valley. Easy to see, hard to walk — it bends while it narrows.

PART III

The Catches — and the Clever Compromise

Newton can walk uphill, and its bend table can be enormous. BFGS fixes both, cheaply.

Newton Can Walk Uphill



WORKED EXAMPLE

$f(x) = x^4/4 - x^2$, start at $x = 0.5$

slope $= -0.875$, bend $= -1.25$

new $x = 0.5 - (-0.875 \div -1.25) = -0.2$

height goes UP: $-0.234 \rightarrow -0.040$ ✗

THE SAFE VERSION

If the bend is negative, or the jump does not go down, take a normal downhill step instead (with Week 5's backtracking).

Every real Newton code has this safety check built in — called 'damped' or 'safeguarded' Newton.

The Bend Table Can Be Huge

THE PROBLEM

With n variables, the bend must be measured for every PAIR of directions: a table of n × n numbers.

variables n	numbers in the bend table	memory
2	4	**nothing**
1,000	1 million	**8 MB**
1,000,000	1 trillion	**8 TB**

WHERE 1,000,000 VARIABLES APPEAR

A small neural network easily has a million weights. Storing its bend table would need about 8,000 GB.

AND IT IS SLOW

Using the table means solving a big system of equations every step — far more work than one slope.

Newton: few, expensive steps. Gradient descent: many, cheap steps. Can we get the best of both?

BFGS: Learn the Bend From Slopes You Already Have

WORKED EXAMPLE

Estimate the bend of $(x - 3)^2$ without any formula

at $x = 7$ the slope was 8; at $x = 5$ the slope was 4

bend ≈ change in slope ÷ change in x

$= (8 - 4) \div (7 - 5) = 2$ ✓ (the true bend)

THE IDEA

Every step gives two slopes for free. Their difference reveals the bend.

BFGS

Builds up the bend table step by step from slope changes. No second derivatives needed.

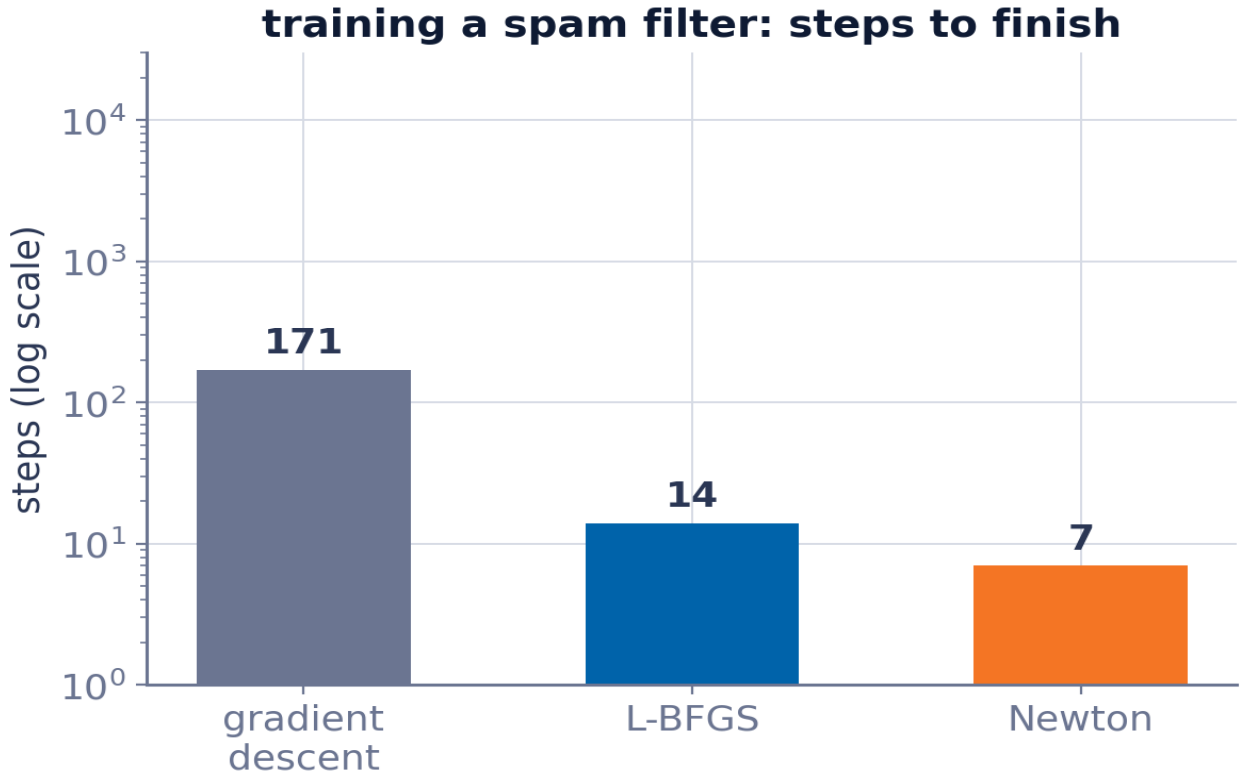
L-BFGS

Remembers only the last ~10 steps. Tiny memory, works with millions of variables.

L-BFGS is the default 'smart' optimizer in most scientific software.

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Training a Spam Filter: 171 vs 14 vs 7 Steps



THE PROBLEM

1,500 emails, 15 features each (words, links, sender…). Learn 15 weights that best separate spam from normal mail.

BUT STEPS ARE NOT TIME

Each Newton step costs much more than a gradient step. With 15 weights all three finish in milliseconds; with millions, only L-BFGS or SGD are practical.

Fewer steps is only a win when each step stays affordable.

Which Method When? A Simple Guide

your problem	use	why
few variables (up to ~1,000), bend easy to compute	Newton (safe version)	**fewest steps, very accurate**
many variables (thousands to millions)	L-BFGS	**almost Newton's speed, small memory**
huge data, neural networks	SGD / Adam (Week 10)	**each step uses only a little data**

```
from scipy.optimize import minimize
result = minimize(f, x0, jac=slope, method="L-BFGS-B")     # one line does it all
```

You rarely write Newton yourself — but knowing how it works tells you which option to pick.

Lab 6: Newton in Ten Lines

```
def slope(x): return x**2 - 2
def bend(x):  return 2*x

x = 1.0
for step in range(10):
    g, b = slope(x), bend(x)
    if abs(g) < 1e-12:      # flat: stop
        break
    x = x - g / b           # the one line
    print(step, x)
```

WHAT YOU WILL DO

1. Reproduce the √2 table. Count the digits.
2. Find √3 — then start at −1 and see the trap.
3. Race gradient descent vs Newton on Week 5's bowl.
4. Call scipy's BFGS on the banana; count its steps.

The full notebook (Week06_Lab.ipynb) has more; sections 1–2 match this lecture.

Six Rules of Thumb

1

Use the bend, not just the slope.

new x = x − slope ÷ bend

2

No step size to guess.

Newton's step is 1 ÷ bend — computed

3

Near the answer, digits double.

√2 to 8 decimals in 4 steps

4

Narrow valleys don't fool it.

139 zig-zag steps became 1

5

Watch for hilltops.

negative bend → take a normal downhill step

6

Big problems: use L-BFGS.

it learns the bend from slopes

WEEK 6 — TAKEAWAYS

1. **One line —** new x = x − slope ÷ bend. That is Newton's method.

2. **One example —** √2: 1 → 1.5 → 1.4167 → 1.414216 → 1.41421356.

3. **One danger —** a negative bend sends Newton uphill — check it.

4. **One compromise —** BFGS / L-BFGS: learn the bend from slope changes.